Actor-Neutral Coordination over Bonded Commitments: An AI-and-Control-Theory Reading

Alessandro Daliana

May 2026

Abstract

Coordination among mutually untrusted counterparties has historically required institutional enforcement: courts, market intermediaries, platform operators, or other vetters that resolve disputes and authorize participation. The control-theory tradition develops mechanisms by which a centralized planner can allocate work under specified objective functions and constraints; the market-design tradition develops mechanisms by which decentralized parties bid for, claim, and settle work without a centralized planner; both have historically assumed the participants are *trusted* in some structural sense — either through the planner's authority or through the market institution's enforcement apparatus. Truly arms-length coordination has remained difficult because the enforcement apparatus has had nowhere to live without re-introducing a centralized party. The same gap blocks coordination among autonomous agents, where the institutional vetter is structurally absent.

We present the bonded settlement primitive — asymmetric bonding plus buyer dominance with atomic resolution — as the missing enforcement layer. The primitive is *actor-neutral* by construction: the kernel makes no distinction between human and algorithmic participants because the kernel reads only EIP-712 signatures and bond posture, not the type of the entity behind a wallet. The equilibrium argument that makes cooperation weakly dominant for human counterparties extends without modification to autonomous agents; humans, agents, and hybrid systems (human-supervised agents, multi-key wallets, agent-supervised humans) participate as co-equal classes of wallet holder. Agent-only coordination is the extreme case the primitive admits, not the rule it was built for.

The paper develops the actor-neutral coordination architecture in three layers: the *kernel* layer (actor-neutral by construction); the *Agent SDK* layer (a stateful loop synchronizing process state, proposing valid actions, and dispatching them through a pluggable policy gateway); and the *factotum* layer (a reference participation agent that wires the SDK to a wallet and a policy and is forked by every operational deployment). The SDK is convenience tooling for wallet-bearing clients; the protocol is the kernel ABI, and any wallet that signs EIP-712 commitments and posts bond participates with or without the SDK. We treat the policy interface as a control-theory specification problem and discuss reference policies for typical operational roles. LLM-augmented policies are a natural specialization for non-trivial decision logic. Discoverability via ERC-8004 and `did:web` is metadata, not protocol.

Keywords: multi-agent coordination, bonded commitment, control-theory, actor-neutrality, autonomous agents, AI agent design, human-in-the-loop

1 Introduction

Coordination among mutually untrusted counterparties is the problem of getting parties who do not share institutional context to cooperate on shared work without producing defections or deadlocks. The classical formulation [Shoham and Leyton-Brown, 2009, Parkes and Seuken, 2024] treats coordination as a mechanism-design problem in which the designer chooses a protocol with desirable equilibrium properties and the participants play within the protocol. The market-design specialization [Milgrom, 2017] treats specific allocation problems (auctions, matching,

scheduling) for which competitive mechanisms are well-studied. The control-theory specialization [Khalil, 2002, Bertsekas, 2005] treats coordination as a distributed-control problem with a designed objective and stability analysis.

Across all three frameworks, the enforcement question is treated asymmetrically with the design question. The mechanism designer specifies the protocol, the participants follow it, and enforcement is either assumed (the participants are honest), structurally provided by the institution running the mechanism (auctioneers, planners, controllers), or external to the mechanism (escrow, court, reputation system). Truly arms-length, institution-free coordination has remained out of reach: the institutional-enforcement assumption persists across these traditions, and is even more visibly absent when the participants are autonomous agents — the institutional vetters that backstop human counterparties have no analogue when the counterparties are software.

The bonded primitive supplies the missing enforcement layer. The bonded settlement primitive's equilibrium properties are obtained without any institutional enforcement: cooperation is the unique profile surviving iterated elimination of weakly dominated strategies in the bonded game, with bonds posted by the parties themselves and resolution executed by the kernel without discretion. The result holds without any assumption about the *type* of the entity behind a participating wallet — the kernel reads EIP-712 signatures and bond amounts, not personhood information — so the equilibrium properties extend to autonomous agents directly. This is not a metaphor; the kernel cannot distinguish a human participant from an algorithmic participant because it does not look.

The actor-neutrality property turns the bonded primitive into a natural enforcement layer for arms-length coordination of any kind. A wallet that bonds against its own performance is playing the bonding game; whether the wallet is held by a human, an autonomous agent, or a hybrid system (human-supervised agent, agent-supervised human, multi-key wallet) is irrelevant to the equilibrium arithmetic. The enforcement question — traditionally the binding constraint on mutually-untrusted coordination design — is dissolved at the substrate layer, leaving the higher-layer questions (policy design, decision logic, work discovery) clear of the enforcement-mechanism complexity that has typically obscured them. Agent-only coordination is the extreme case the architecture admits, not the rule it was built for.

Paper organization. Section 2 summarizes the settlement primitive in the register the present paper uses. Section 3 develops actor-neutrality as a formal property of the kernel and traces its consequences for agent coordination. Section 4 presents the Agent SDK as a control-theory architecture: a stateful loop with sync, propose, policy, and execute phases. Section 5 treats policies as controllers and develops reference policies for typical operational roles. Section 6 treats LLM-augmented policies and the latency / non-determinism trade-offs they introduce. Section 7 treats discoverability via ERC-8004 and `did:web` as metadata-layer infrastructure that does not affect kernel-layer enforcement. Section 8 states the scope conditions and the security posture the architecture invites. Section 9 concludes.

2 The Settlement Primitive

The bonded primitive's mechanism-design derivation and its formal verification (TLA$^+$, Halmos symbolic execution, Echidna property-based fuzzing, Certora specification rules) are out of scope for the present paper. The features the agent architecture relies on are the following.

Asymmetric bonding (Mechanism 1). For a transaction with payment $P > 0$ and cumulative upstream value $G \geq P$, the buyer locks $2P$ and the seller locks $2G$. Cooperation is

the unique profile surviving iterated elimination of weakly dominated strategies. The mechanism scales to N-party process chains through progressive collateralization; the organizational topology over which orders compose is an upper-layer concern.

Buyer dominance with atomic resolution (Mechanism 2). Only the root buyer can trigger resolution; resolution settles all active orders simultaneously or not at all. The atomicity rule induces a weakest-link subgame among sellers, propagating cooperation pressure of magnitude $P_i + 2G_i$ on every other seller without explicit communication.

What this means for an agent. An autonomous agent participating in a bonded commitment faces the same payoff structure as a human participant. The agent's bond is exposed to the kernel-rule resolution exactly as a human's bond is. The agent's defection is dominated by cooperation under the same arithmetic that makes a human's defection dominated. The agent's expected payoff under cooperation is $+P$ (seller side) or $-P$ (buyer side); under defection it is the bond loss. There is no agent-specific incentive structure. The kernel has no API for "trust this party more because they are an agent" or "trust them less." The kernel has no notion of party type.

3 Actor-Neutrality

We state and discuss the actor-neutrality property formally.

Definition 3.1 (Actor-Neutrality of the Kernel)**.** A coordination kernel is *actor-neutral* if its operational behavior depends only on the cryptographic facts visible to the kernel (EIP-712 signatures, bond posting, agreement-hash, process state) and not on any out-of-band classification of the entity behind a wallet (human vs. agent, individual vs. institutional, identified vs. pseudonymous, jurisdiction-of-residence, regulatory status). Specifically, the kernel's transition function is invariant under any consistent re-labeling of the parties as the "same wallet, different entity-type behind it."

The Figaro kernel is actor-neutral. The kernel's two operations (`commit` and `resolveProcess`) consult only the cryptographic facts listed in the definition. There is no wallet-type registry; the `OperatorRegistry` contract carries advisory metadata at the runtime tier but is not consulted by the kernel for any state-changing operation. Role and metadata travel with the `OperatorRegistered` event; the contract itself stores only a dedup guard and a deposit-lock timestamp. There is no party-type input parameter; the kernel's signatures are over typed agreements, and the typed agreement does not carry party-type information. The `commit` function verifies signatures, transfers bond amounts, and updates process state; none of these steps reads party-type information. The `resolveProcess` function verifies the caller is the root buyer, verifies all active orders are present, and executes the bonding rule's payouts; none of these steps reads party-type information.

Why this matters for agent coordination. The standard architectural objection to mutually-untrusted agent coordination is that agents need to be vetted, certified, or otherwise distinguished from un-vetted agents before they can be trusted to interact with sensitive operations. The vetting infrastructure is itself an enforcement-apparatus question, and deploying it requires deciding which authority does the vetting, how the authority is held accountable, and what happens when an agent is mis-classified. The vetting apparatus is the institutional layer the bonded primitive eliminates.

Under actor-neutrality, vetting is unnecessary at the kernel layer. An un-vetted agent that bonds against its own performance has the same incentive structure as a vetted agent that

bonds against its own performance. The agent's bond is the vetting; performance is enforced by the bonding rule rather than by prior assessment of the agent's reliability. This collapses the authority-of-the-vetter question into the architectural fact of bond posting.

Remark 3.2 (Vetting moves up the stack). Actor-neutrality at the kernel does not mean vetting is irrelevant generally. Counterparties that prefer to interact only with vetted agents can read the `OperatorRegistry` metadata, the agent's settlement history (visible on chain), or any external attestation registry the parties choose to consult, and decline to sign commitments with agents that fail their criteria. The kernel-layer enforcement is unconditional on agent type; the counterparty-side selection is a runtime-tier choice that is expressible in any specific assembly the parties compose.

The same equilibrium for the agent. The equilibrium properties of Section 2 hold for any party in the bonded game, regardless of whether the party is human or autonomous. The dominance argument is constructed from the bonding arithmetic alone; no appeal to party-type enters the proof. An autonomous agent that defects forfeits its bond; the agent's expected payoff under defection is therefore $-2G$ (seller side) or $-2P$ (buyer side), exactly the same as a human's. Cooperation is weakly dominant; the agent's optimal play in the bonded game is the same play a human's optimal play would be.

This collapses the "trust the agent" question into the bonding arithmetic. A counterparty considering whether to bond with an autonomous agent does not need to certify the agent's reliability; the bond's existence is the certification. Whether the bond is posted by an LLM controller, a deterministic rule-based policy, a hybrid system, or a human acting through an agent's wallet is irrelevant to the kernel and irrelevant to the equilibrium.

4 The Agent Loop

We now present the Agent SDK as a reference architecture for wallet-bearing clients participating in the bonded primitive. The SDK is convenience tooling, not a protocol requirement: the protocol is the kernel ABI, and any wallet that signs EIP-712 commitments and posts bond participates with or without the SDK. The SDK matters because it standardizes a shape that operational deployments converge on regardless. The architecture is a stateful loop with four phases: sync, propose, policy, execute.

The control-theory framing. Read in control-theory terms, the wallet's policy is the controller, the chain is the plant, and the policy's decision is the control law. The state of the plant is the on-chain process graph: the set of active processes, their committed orders, their attestations, and their settlement events. The agent observes the state through its sync phase, proposes admissible actions through its proposer, and selects an action through its policy. The execute phase commits the selected action to the chain, advancing the plant state. The loop runs until shut down, with the agent's wallet as the identifier under which actions are committed.

Sync. The sync phase reconstructs the process graph from on-chain events. The reference implementation (`FigaroContext` in `@figaro/core/agent`) maintains a local `ProcessGraph` that ingests `OrderCommitted` and `OrderResolved` events from the kernel contract. Sync is incremental: the first call fetches from the kernel's deployment block; subsequent calls fetch only new blocks. Sync produces a `SyncResult` carrying the count of new commits, new resolutions, and the synced-to-block. The graph is the agent's view of the plant state.

There is no subgraph requirement, no indexer dependency, and no API gateway between the agent and the chain. The sync phase reads events directly. This is operationally important for the trust-minimization story: the agent's view of the world is reconstructed from the same

cryptographically-verifiable on-chain state that any other party can verify, with no intermediation that could distort the view.

Propose. The propose phase is a pure function over process state: `proposeActions(process, myAddress)` returns the actions admissible to the address `myAddress` given the current process state. The action types are `commit-sub-order` (extending the process chain), `resolve-process` (the buyer-dominance settlement), `attest-as-seller`, and `attest-as-buyer`. Each `ProposedAction` carries a human/LLM-readable description, pre-validated parameters, and bond math (approval amounts, payouts).

The proposer is pure: no side effects, no signing, no submission. The agent (human or autonomous) decides which proposed action to execute through the policy phase. The proposer's role is to narrow the space of all syntactically-valid kernel transactions to the space of process-state-admissible actions for the given address; the policy's role is to select among those.

Policy. The policy is the agent's decision logic. The policy interface is batch-shaped: a policy decides on a list of pending action entries and returns a list of approve/reject decisions.

```
interface Policy<TAction, TContext = unknown> {
  name: string;
  decide(
    entries: PolicyEntry<TAction, TContext>[],
  ): Promise<PolicyDecision<TAction, TContext>[]>;
}
```

The factotum reference implementation supplies the policy primitives at `agents/factotum/src/policy.ts` — `makeHitlPolicy()`, which returns a human-in-the-loop policy that defers every action to operator approval, and `makeAutonomousPolicy(shouldExecute)`, which returns an autonomous policy whose per-action decision is delegated to a caller-supplied `(action, ctx) => {execute, reason?}` callback. Role-shaped reference policies ship as separate factories at `agents/factotum/src/policies/` — `auditorPolicy`, `basicMerchantPolicy`, `buyerWithBudgetPolicy`, `courierBidderPolicy`, and `sellerOfRecordPolicy`, each composing the primitives above with role-specific parameters. Operational deployments either fork one of these or compose the primitives directly; we treat typical roles in Section 5.

Execute. The execute phase commits an approved action to the chain via the agent's wallet. The SDK exposes two execution surfaces. For self-contained actions whose parameters are fully captured on the `ProposedAction`, `executeAction` in `@figaro/core/agent` builds and submits the transaction directly via a viem `WalletClient`; in the current SDK this covers `resolve-process`. For actions whose execution requires inputs beyond what the proposer carried (signed commitments, attestation `sectionData`, merkle proofs), the SDK exposes standalone direct functions — `commit`, `attestAsSeller`, `attestAsBuyer` — which operational deployments call after assembling the additional inputs at submission time. A separate `SequencerClient` export covers the batched-execution path for deployments that prefer to consolidate multiple actions into a single sequencer submission rather than direct on-chain calls. Execution on either surface is sequential per agent: actions are committed one at a time, with the agent's nonce advancing per transaction. Parallel execution would race the kernel state and is explicitly avoided.

The full loop. A factotum (the reference participation agent at `agents/factotum/`) wires the four phases together with a polling interval, loading the wallet from configuration, selecting the policy from runtime arguments (HITL by default), and running the loop until shutdown. The factotum is not a production system; it is a fork-and-modify reference that any operational deployment specializes into its own runtime. Other runtimes (Python, Rust, Go) are expected;

the protocol is language-agnostic, and the only requirements are a wallet, EIP-712 signing, and event-derived state.

5 Policies as Controllers

Read in control-theory terms, the policy is the wallet's control law. The space of admissible policies is wide; the architecture constrains the policy interface but not the policy logic. We develop reference policies for typical operational roles. The roles are not agent-specific; the same policies apply when the wallet is held by a human operator approving each action through a UI, by an autonomous agent running its own decision logic, or by a hybrid system in which an autonomous proposer routes high-stakes actions to a human reviewer.

Human operator with HITL approval. The simplest policy under the architecture is the one a human operator running the SDK from a web UI follows. Every proposed action is presented to the operator through an `ActionQueue`; the operator approves or rejects each one, and the SDK executes only the approved entries. The shipping `makeHitlPolicy()` factory at `agents/factotum/src/policy.ts` returns exactly this policy: every entry is queued for operator review; nothing is auto-approved. A wallet driven only by `makeHitlPolicy()` is operationally indistinguishable from a wallet driven by a person clicking "confirm" on every transaction. The kernel sees the same EIP-712 signatures and bond posting either way. This is the trivial case actor-neutrality permits and the case many production deployments will run for high-value processes.

Buyer-with-budget. A buyer in a bonded process commits sub-orders that contribute to the cumulative process value and resolves the process when performance attestations indicate completion. A buyer-with-budget policy bounds the buyer's exposure by a per-commit cap and a total-budget cap. Two operational facts about the SDK's action shape inform the policy: `action.currentCumulativeValue` exposed by `proposeActions` carries the cumulative-prior value (the sum of all previously-committed orders' payments), not the new sub-order's payment, and the new sub-order's payment is supplied by the off-chain agreement the agent is signing against rather than as a field on the proposed action. A buyer-with-budget policy therefore reads from `action.currentCumulativeValue` together with a deployment-supplied tracker of per-process spend; the shipping `buyerWithBudgetPolicy` factory at `agents/factotum/src/policies/buyerWithBu` composes the primitive `makeAutonomousPolicy` with this pattern:

```
const PER_COMMIT_LIMIT = 1_000_000_000n; // $1K
const TOTAL_BUDGET = 50_000_000_000n;   // $50K
const buyerPolicy = buyerWithBudgetPolicy({
  perCommitLimit: PER_COMMIT_LIMIT,
  totalBudget: TOTAL_BUDGET,
  budgetSpent,
});
```

where `budgetSpent` is a mutable cell of shape `{ value: bigint }` holding the running total of bonded commits the deployment has executed under this policy. The factory's internal `shouldExecute` callback consults `action.currentCumulativeValue`, applies the per-commit and total-budget limits against `budgetSpent.value`, and approves `resolve-process` actions unconditionally; the deployment owns the cell and is responsible for incrementing it on successful execution.

The control-theory reading: the policy enforces a budget constraint on the closed-loop trajectory of bonded commits, with the constraint bounding the cumulative bonded exposure across the process.

Seller-of-record. A fan-out actor (an airline, an ocean carrier, a marketplace) acts as seller to the buyer and as buyer at every sub-procurement. By volume this is typically the busiest factotum in any non-trivial assembly. A split policy is the right shape: autonomous for routine attestations and small sub-procurements, HITL for resolutions and exceptional sub-procurements:

```
const policy = sellerOfRecordPolicy({
  routineProcurementLimit: 5_000_000_000n, // $5K
});
```

The shipping `sellerOfRecordPolicy` factory at `agents/factotum/src/policies/sellerOfRecord.ts` approves attestation actions (`attest-as-seller`, `attest-as-buyer`) autonomously and approves `commit-sub-order` actions whose cumulative value falls under the configured `routineProcurementLimit`; everything else is rejected with a reason and escalates to the operator's HITL surface. An operational deployment configures the routine-limit threshold to match its operational reserves.

Dutch-auction bidder. A bidder on a Dutch auction (the protocol's `DutchAuction` mechanism contract) monitors a descending price curve and claims the position when the price meets a profitability threshold. A subtlety of the current SDK surface deserves naming: `proposeActions` as currently implemented emits `commit-sub-order` only for the address that holds the buyer-side role in the process chain, not for prospective sellers competing on a Dutch auction. A factotum acting as a prospective Dutch-auction-bidder therefore does not receive a `commit-sub-order` proposal directly; the bidder's work-discovery happens through the `DutchAuction` mechanism contract's own auction-state events, surfaced off-chain through the `@figaro/core/extensions` auction helpers (`computeCurrentPrice`, `evaluateClaim`). The policy acts at the moment the bidder is preparing a claim transaction rather than at the moment a kernel-level action is proposed. A reference `courierBidderPolicy` ships at `agents/factotum/src/policies/courierBidder.ts` as the shipping example of this pattern. The bidder's logic, expressed as a pre-claim gate rather than as a factotum policy in the strict SDK sense, looks like:

```
const MIN_MARGIN_BPS = 500n; // 5% margin
const dutchAuctionBidderGate = (
  auction: AuctionState,
  ctx: Ctx,
) => {
  const offered = auction.currentPrice;
  const myCost = ctx.estimateMyCost(auction);
  const marginBps = ((offered - myCost) * 10000n) / myCost;
  return marginBps >= MIN_MARGIN_BPS
    ? { claim: true }
    : { claim: false, reason: "below margin threshold" };
};
```

The control-theory reading: the policy implements a threshold-rule controller on the price-vs-cost gap, with the threshold parameterized by minimum acceptable margin. The agent does not optimize across competing auctions; it claims the first auction that crosses its threshold, leaving cross-auction optimization to a higher-layer policy if the deployment wants one. The bidder gate runs adjacent to the factotum loop rather than inside it, reflecting the architectural split: kernel-level actions live in `proposeActions`; auction monitoring and evaluation live in the extensions surface; the bidder composes both at the deployment layer.

Auditor. A passive observer that emits attestations against process events without committing or resolving. Auditor policies accept attestation actions and refuse all others:

```
const auditorPolicy = makeAutonomousPolicy<ProposedAction, Ctx>(
  (action) => {
    if (action.type === "attest-as-seller" || action.type === "attest-as-buyer")
```

```
      return { execute: true };
    return { execute: false, reason: "auditor does not commit" };
  },
);
```

The default refuse-all behavior. The architecture's policy interface defaults to refuse-all when no rule is supplied. A fresh-fork factotum running in autonomous mode with no rule does nothing on chain. This is by design: the security floor is that an unconfigured agent cannot accidentally commit, resolve, or attest. The operator must explicitly write the rule that authorizes any specific action type. The architecture chooses safe-by-default over convenient-by-default.

6 LLM-Augmented Policies

The recent wave of LLM-agent architectures [Yao et al., 2023, Shinn et al., 2023, Park et al., 2023, Wu et al., 2023] treats an LLM as the decision substrate of an autonomous agent: the LLM observes context, reasons over it, and selects tool calls. The policy interface developed here is LLM-agnostic: any synchronous-or-asynchronous function over actions is admissible. Wrapping an LLM call inside `shouldExecute` is direct, and is appropriate for decisions whose logic is non-trivial, context-dependent, or requires natural-language reasoning.

Two cautions. LLM latency is real. A 2-second model call inside a tight loop limits the agent's throughput, and inside an auction context can produce a miss when a faster competitor's policy claims the auction during the LLM's deliberation. The architectural responses are: cache decisions where the input space repeats, batch the LLM context across multiple actions where the LLM can usefully reason about a set, and move the LLM out of the hot path for actions whose latency is critical (routine attestations, time-sensitive auction claims). The hot-path decisions stay in deterministic policy logic; the LLM-augmented decisions are reserved for actions where a longer deliberation is acceptable.

LLM decisions are non-deterministic. Two identical inputs may produce different verdicts on different runs. For high-value actions (commits, resolutions) the recommendation is to keep human-in-the-loop and route LLM verdicts to a separate review layer rather than to direct execution. For low-stakes filters (which jobs to surface, which to ignore, when to escalate to a human reviewer) LLM autonomy is operationally tractable.

Prompt injection as a defection vector. LLM controllers consume context that may include adversarial input: attestation content surfaced through the proposer, off-chain agreement text, third-party-supplied operational notes [Greshake et al., 2023]. A poisoned context can steer the LLM into approving actions the operator did not authorize. The defense is structural: the deterministic policy layer applies hard limits (per-commit cap, total-budget cap, allow-list of counterparties, schema-typed action validation) that the LLM's verdict cannot override. The bond posture amplifies the defense: even if the LLM is steered into approving a bonded commitment the operator would not have approved, the loss is capped at the bond that commitment entails, not at the operator's full treasury. The combination of hard policy limits plus per-action bond capping makes the LLM's worst case operationally analyzable rather than catastrophic.

Tool-use frameworks and the policy interface. Tool-use frameworks (function-calling APIs, the Model Context Protocol, the Agent-to-Agent protocol) select among actions an LLM can take in a given context. The policy interface developed here is composable with these frameworks at one site: the deterministic policy gate runs after the LLM proposes an action through

its tool-use surface and before the SDK executes it. The LLM's tool-use selection becomes a recommendation; the policy remains the authority on what reaches the chain. Operational observability [Dong et al., 2024] is the third leg: proposed actions, policy decisions, executed transactions, and post-execution divergence between expected and observed state are all logged for review and replay.

The hybrid pattern. A natural hybrid wires deterministic-rule policy as the front line and LLM as the second-stage filter. The deterministic rule disposes of routine actions (attestations, sub-procurement under threshold, default-resolve when all attestations are clean); the LLM is consulted for actions the rule defers (exceptional sub-procurement, ambiguous attestation patterns, resolution decisions on contested processes). The hybrid bounds the LLM's hot-path exposure while still capturing its value on the genuinely-novel decisions.

Why bonding is the safety floor (and why it is not, by itself, sufficient). LLM-augmented policies are dangerous to deploy in unsupervised settings if the agent's decisions are unrecoverable. The bonded architecture supplies a per-action bound: an LLM-augmented agent's maximum loss on any single action is bounded by the bond exposed at that action, so a misjudged commit forfeits the bond at that sub-process and not the agent's entire treasury. This is a real structural property and it is what the kernel's bonding rule gives for free.

The per-action bound does not by itself produce a treasury-level bound. An agent operating across many concurrent processes with small per-action bonds still faces aggregate loss exposure equal to the sum of active bonds, which scales with the number of processes the agent has open. An LLM that mis-deploys across N concurrent processes at a per-process bond of B faces an aggregate exposure of $N \cdot B$, which grows unbounded in N unless an aggregate cap is imposed. The kernel does not impose the aggregate cap; the policy does. The buyer-with-budget reference policy of Section 5 is one such cap (the `TOTAL_BUDGET` parameter); a deployment that omits the cap or sets it too high inherits the unbounded-aggregate exposure.

Two distinct bounds therefore matter for LLM-augmented operation: the per-action bound (kernel-level, automatic, equal to the bond posted at that action) and the aggregate bound (policy-level, operator-imposed, equal to the cap on cumulative bonded exposure across active processes). The architecture supplies the first; the operator must supply the second through policy code or aggregate-budget tracking. Both are required for autonomous deployment at scale.

This is a structural advantage relative to LLM agents operating outside the bonded primitive. An LLM that can autonomously execute arbitrary transactions on its operator's behalf has loss exposure bounded only by the operator's wallet balance and the operator's willingness to refill it. An LLM operating through a factotum on the bonded primitive has the kernel-level per-action bound for free, plus a cleanly defined site (the policy's aggregate-cap field) at which the operator imposes treasury-level bounds. The bonded architecture makes LLM-policy risk decomposable into analyzable components rather than collapsed into a single unbounded surface; it does not eliminate the need for operator-imposed limits.

7 Discoverability

Discoverability — how agents *find* each other across an open ecosystem — is metadata, not protocol. The bonded primitive's enforcement does not depend on discoverability; bonded counterparties who already know each other can transact without any discovery layer. Discoverability becomes important at scale, when agents need to identify suitable counterparties they have not previously interacted with. The architecture provides a metadata-layer solution that does not require kernel-layer changes.

ERC-8004 and what it actually covers. ERC-8004 [ERC-8004, 2026] (Trustless Agents, finalized and live on Ethereum mainnet 2026-01-29) is a three-registry standard:

- *Identity Registry*: portable agent identifiers backed by ERC-721 NFTs.
- *Reputation Registry*: bounded numerical scores and categorical tags for agent feedback (response time, uptime, etc.).
- *Validation Registry*: task-verification mechanisms via TEE attestations, zkML proofs, or crypto-economic staking.

The architecture this paper presents replaces two of the three registries by construction and complements the third. Bonded performance *is* the validation: the bond a participant posts at `commit` is precisely the crypto-economic stake the Validation Registry asks for, applied at the per-order level rather than per-agent. On-chain settlement history *is* the reputation evidence: a resolved process is a stronger signal than any bounded numerical score, because it is the immutable record of bonded performance under economic stakes. The Reputation and Validation registries' purpose is satisfied at the kernel layer without a separate registry surface.

The Identity Registry is the layer that does not collapse into the bond. Cross-organization identifiers, capability descriptors, and service endpoints (MCP, A2A) live above the bond and serve a different function: helping a wallet *find* a counterparty that has not yet been bonded with. The architecture treats this as an off-chain concern. The `OperatorRegistry` contract carries a `metadataURI` per registered operator, pointing at an off-chain JSON document; the convention developed here aligns the document with ERC-8004 identity claims and the W3C `did:web` method [W3C, 2022]:

```
{
  "subjectAddress": "0xYourAgent",
  "archetypeId": "autonomous-driver",
  "services": {
    "did": "did:web:agent-42.example.com",
    "mcp": "https://agent-42.example.com/mcp",
    "a2a": "https://agent-42.example.com/a2a"
  },
  "capabilities": ["route-optimization", "live-eta"]
}
```

The SDK provides `resolveDidWeb()`, `didDocumentMatchesAddress()`, and `buildOperatorDidDocument()` helpers in `@figaro/core/extensions`. The metadata document is the extension point at which ERC-8004 identity descriptors can be adopted by any deployment that wants them; no kernel-layer contract change is required.

Why agents are wallet-tier, not entity-tier. A consequence of the actor-neutrality property is that the architecture does not register agents as first-class on-chain entities. ERC-8004's Identity Registry takes the alternative route: each agent is a registered entity with its own NFT-backed handle. Adopting that registry as a kernel-level requirement would re-introduce the platform-as-vetter problem the actor- neutrality property is designed to eliminate. A wallet-tier agent is the consistent extension of kernel-level actor-neutrality one tier up: the wallet is the agent's identity, and the agent's reputation is its on-chain settlement history. ERC-8004 identifiers can travel in the metadata document for deployments that want them, but they are optional discoverability metadata, not a registration the architecture requires.

The structural separation. Identity descriptors (whether ERC-8004-formatted or otherwise) are how wallets *find* each other; bonded commitments are how they *trust* each other. The two are intentionally separate. A wallet that publishes service descriptors is discoverable but not yet

trusted; a counterparty that wants to interact reads the descriptors, evaluates the self-published capabilities, and then bonds against performance. The bond is the trust layer; the descriptor is the discovery layer. Conflating the two — by, for instance, requiring discovery to imply trust or by requiring trust to imply discovery — would re-introduce the platform-as-vetter problem the actor-neutrality property eliminates. The architecture keeps them separate.

Capability claims are unverified at the kernel layer. The `capabilities` array in an agent's metadata is *self-asserted*. The kernel does not verify that an agent who claims `"route-optimization"` can in fact optimize routes; the kernel's enforcement is on bonded performance, not on capability claims. A counterparty that bonds against an agent's performance is exposed only to bond loss if the agent under-performs; the counterparty's view of the agent's capability is informed by the agent's prior on-chain settlement history (which the counterparty can read directly from the chain) and by any external attestation the counterparty chooses to consult, but is not certified by any component of the protocol. This is the appropriate posture for permissionless infrastructure: capability claims are signals; performance is enforcement.

8 Scope and Security Posture

The architecture is bounded by several scope conditions and operational constraints that any deployment must respect.

Hardware-isolated signing for production. A long-running agent has wallet access by design. Hot-key compromise is catastrophic in proportion to the agent's bond posture. Production deployments use hardware-isolated signers (HSMs, secure enclaves, multi-sig with hardware-key co-signers) rather than software keys loaded into the agent's runtime. This is standard guidance for any high-value autonomous wallet, but the bonded architecture amplifies its importance because the agent's exposure can be substantial.

Default-HITL for high-value actions. The architecture's default policy is human-in-the-loop. Autonomous mode is for actions whose worst case is bounded by predictable parameters (a single auction claim under $X; a routine attestation that reflects an externally-observed fact). For commits, resolutions, and any action affecting other parties' bonds, the recommended posture is HITL until the operator has accumulated sufficient operational confidence in the autonomous policy. Migration from HITL to autonomous is a deployment decision that follows from observed policy behavior across many decisions, not a default the architecture recommends.

Defense-in-depth in the policy. The proposer is correct by construction; the policy should still defend against unexpected proposer behavior as a defense-in-depth layer. Re-derive bond amounts from process state in the policy. Re-check counterparty addresses against expected sets. Bound gas in the execute layer. The policy is the operator's last line of defense against any architectural assumption that fails in operational context.

Logging and observability. Settlement disputes off-chain need an audit trail; the chain has half the story. An operational deployment logs proposed actions, policy decisions, executed transactions, and any divergence between the agent's expected and observed state. Off-chain forums adjudicating bonded commitments treat the on-chain settlement record as evidentiary input; the agent's audit trail is the operational complement on the agent's side. This is standard operational practice for any autonomous system; the bonded architecture invites it because the agent's actions have material consequences that may be reviewed off-chain.

What this is not. The architecture is not a coordination strategy. The factotum executes what the proposer suggests within the policy's authorization; it does not decide which markets to enter, which prices are profitable, which counterparties to trust at the discovery layer, or which strategic objectives to pursue. The strategy lives in the policy and in the deployment's higher-layer orchestration. The architecture is the substrate; the strategy is the operator's.

The architecture is also not LLM-specific. The policy interface admits any decision-logic. LLM-augmented policies are one specialization; rule-based policies, reinforcement-learning policies, model-predictive-control policies, and human-driven policies are equally well supported. The architecture's LLM-friendliness derives from the policy interface's generality, not from any LLM-specific design.

9 Conclusion

Coordination among mutually-untrusted counterparties has been hard because the enforcement layer has had nowhere to live without a centralized institution; the same gap blocks coordination among autonomous agents, where the institutional vetter is structurally absent. The bonded settlement primitive's actor-neutrality property turns bonded commitments into the missing enforcement layer: the same equilibrium argument that makes cooperation weakly dominant for any participant makes it weakly dominant regardless of whether the participant is a human, an autonomous agent, or a hybrid system. The bond posture is type-blind by construction.

The Agent SDK presented in this paper is a reference architecture for wallet-bearing clients: a stateful loop with sync, propose, policy, and execute phases, structured as a control-theory controller-on-plant architecture. The SDK is convenience tooling, not a protocol requirement. Reference policies cover typical operational roles (HITL operator, buyer-with-budget, seller-of-record, Dutch-auction bidder, auditor) and apply identically when the wallet is held by a human approving each action through a UI, by an autonomous agent running its own decision logic, or by a hybrid system. LLM-augmented policies are a natural specialization for non-trivial decision logic, with the bonded architecture's per-action exposure bound and the policy's aggregate-cap mechanism turning LLM-policy risk into a decomposable and analyzable surface. Discoverability via ERC-8004 identity descriptors and `did:web` is metadata, not protocol; the discovery layer and the trust layer are separate by design, and ERC-8004's reputation and validation registries are subsumed by the bonded primitive at the kernel layer.

The contribution is architectural rather than novel mechanism design: actor-neutrality is the load-bearing claim, and the SDK is the operational artifact that demonstrates the claim is implementable. The participants are not protocol-aware in any privileged sense; they sign, they bond, they perform. The substrate makes their participation legible to their counterparties and economically rational for them to undertake. Agent-only coordination is the extreme case the architecture admits; coordination between humans, between agents, and across the human-agent boundary is the rule.

Acknowledgments

The Agent SDK and the factotum reference were developed alongside the kernel and the runtime; they are operational realizations of the actor-neutrality property the bonded settlement primitive carries by construction.

References

Shoham, Y. and Leyton-Brown, K. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, Cambridge, 2009.

Parkes, D. C. and Seuken, S. *Economics and Computation: An Introduction to Algorithmic Game Theory, Computational Social Choice, and Fair Division*, 2nd edition. Cambridge University Press, Cambridge, 2024.

Milgrom, P. *Discovering Prices: Auction Design in Markets with Complex Constraints*. Columbia University Press, New York, 2017.

Khalil, H. K. *Nonlinear Systems*, 3rd edition. Prentice Hall, Upper Saddle River, NJ, 2002.

Bertsekas, D. P. *Dynamic Programming and Optimal Control*, 3rd edition. Athena Scientific, Belmont, MA, 2005.

W3C. Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations. W3C Recommendation, July 2022.

De Rossi, M., Crapis, D., Ellis, J., and Reppel, E. ERC-8004: Trustless Agents. Ethereum Improvement Proposal, finalized 2026. `https://eips.ethereum.org/EIPS/eip-8004`

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proc. International Conference on Learning Representations (ICLR)*, 2023.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative Agents: Interactive Simulacra of Human Behavior. In *Proc. ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint, 2023.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

Dong, L., Lu, Q., and Zhu, L. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285, 2024. `https://arxiv.org/abs/2411.05285`